C++ · ALGORITHMS & DATA STRUCTURES REFERENCE

Sorting Algorithms & Custom Sorting in C++

A production-grade reference spanning every sorting algorithm worth knowing: elementary $O(N^2)$ sorts, the efficient O(N log N) comparison sorts, the linear-time non-comparison sorts, and the hybrids real standard libraries actually ship — thirteen algorithms total, each with working C++17/20 code, complexity analysis explaining *why* each bound holds, the exact pitfalls that cause segfaults and overflow in interviews and production, and the Strict Weak Ordering rule every custom comparator must obey.

13 algorithms, cheat sheet included

Function pointers, functors & lambdas

Strict Weak Ordering explained

Multi-criteria struct sorting

Updated Oct 2026

CONTENTS

01 Executive Summary / Cheat Sheet 2

02 Bubble Sort 4

03 Selection Sort 6

04 Insertion Sort 9

05 Shell Sort 11

06 Merge Sort 14

07 Quick Sort (Randomized) 18

08 Heap Sort 22

09 Counting Sort 26

10 Radix Sort 29

11 Bucket Sort 32

12 std::sort 35

13 std::stable_sort 37

14 Timsort (Conceptual Overview) 39

15 Custom Comparators & Strict Weak Ordering 41

16 Sorting Custom Data Structures 44

17 Universal Edge Cases Checklist 46

01 Executive Summary / Cheat Sheet

The comparison matrix interviewers expect you to have memorized cold. Every row is justified in its own section below — don't just memorize this table, understand *why* each cell is what it is. Grouped by family: elementary $O(N^2)$ sorts, efficient comparison sorts, linear-time non-comparison sorts, then production hybrids.

ALGORITHM	BEST	AVERAGE	WORST	AUX. SPACE	STABLE	IN-PLACE
Bubble Sort	O(N)§	$O(N^2)$	$O(N^2)$	O(1)	**Yes**	**Yes**
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	O(1)	**No¶**	**Yes**
Insertion Sort	O(N)§	$O(N^2)$	$O(N^2)$	O(1)	**Yes**	**Yes**
Shell Sort	O(N log N)	~$O(N^{1.25})$•	$O(N^2)$•	O(1)	**No**	**Yes**
Merge Sort	O(N log N)	O(N log N)	O(N log N)	O(N)	**Yes**	**No**
Quick Sort (Randomized)	O(N log N)	O(N log N)	$O(N^2)$‡	O(log N)	**No**	**Yes**
Heap Sort	O(N log N)	O(N log N)	O(N log N)	O(1)	**No**	**Yes**
Counting Sort	O(N + K)	O(N + K)	O(N + K)	O(N + K)	**Yes**	**No**
Radix Sort	O(d(N + b))	O(d(N + b))	O(d(N + b))	O(N + b)	**Yes**	**No**
Bucket Sort	O(N + K)	O(N + K)‖	$O(N^2)$‖	O(N + K)	**Yes·**	**No**
`std::sort`	O(N log N)	O(N log N)	O(N log N)	O(log N)	**No**	**Yes**
`std::stable_sort`	O(N log N)	O(N log N)	$O(N \log^2 N)$†	O(N)	**Yes**	**No**
Timsort	O(N)§	O(N log N)	O(N log N)	O(N)	**Yes**	**No**

§ With the standard early-exit / already-sorted-run optimization. ¶ Selection sort's classic swap-based implementation is not stable (Section 3). • Shell Sort's average/worst bounds depend entirely on the chosen gap sequence — figures shown are for the original (N/2, N/4, ...) sequence; better sequences (Hibbard, Sedgewick) improve the worst case toward $O(N^{3/2})$. ‡ Randomized pivot selection makes the true $O(N^2)$ worst case astronomically unlikely for any input an adversary can construct without knowing your RNG seed — but it is not impossible. ‖ Assumes a roughly uniform input distribution; an adversarial distribution that fills one bucket degrades to whatever the per-bucket sort does. · Stable only if the per-bucket sort used is itself stable (e.g. insertion sort). † `std::stable_sort` guarantees O(N log N) *only* when it can allocate the O(N) auxiliary buffer it wants; if allocation fails it falls back to an in-place merge, degrading to $O(N \log^2 N)$.

HOW TO READ THIS TABLE IN AN INTERVIEW

Don't recite cells — recite the **trade-off** each row represents. The elementary sorts (Bubble/Selection/Insertion/Shell) trade asymptotic efficiency for simplicity, O(1) space, and genuinely good performance on tiny or nearly-sorted inputs. Merge/Quick/Heap trade between extra space, stability, and worst-case guarantees to all reach O(N log N) by different roads. Counting/Radix/Bucket trade generality — they only work when the keys have exploitable structure (bounded integers, fixed digit-width, a known distribution) — for linear time, sidestepping the O(N log N) comparison-sort lower bound entirely. `std::sort`, `std::stable_sort`, and Timsort are the real engineering answers: hybrids that combine several of the above to get the best constant factor on real-world data while keeping a provable worst case.

---

02 Bubble Sort

The simplest sort to describe correctly, and the one most often implemented *without* its one meaningful optimization.

Overview & Working Principle

Bubble sort repeatedly scans the array, comparing each pair of **adjacent** elements and swapping them if they're out of order. Each full pass guarantees the largest remaining element "bubbles" all the way to its correct position at the end — so after `k` passes, the last `k` elements are guaranteed correctly placed and don't need re-checking. The critical optimization: track whether *any* swap happened during a pass — if none did, the array is already sorted and the algorithm can stop immediately, which is what gives it a genuine O(N) best case rather than always running the full $O(N^2)$.

C++ Production Code

```cpp
// C++ program to implement Bubble Sort
#include <bits/stdc++.h>
using namespace std;

// An optimized version of Bubble Sort
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        // If no two elements were swapped, then break
        if (!swapped)
            break;
    }
}

// Function to print a vector
void printVector(const vector<int>& arr) {
    for (int num : arr)
        cout << " " << num;
}

int main() {
    vector<int> arr = { 64, 34, 25, 12, 22, 11, 90 };
    bubbleSort(arr);
    cout << "Sorted array: \n";
    printVector(arr);
    return 0;
}
```

Source: GeeksforGeeks — Bubble Sort Algorithm, reproduced verbatim.

COMPLEXITY ANALYSIS

Best: O(N) — only with the early-exit flag above. On an already-sorted array, the first pass makes zero swaps, `swapped` stays `false`, and the function returns after one O(N) scan. **Average / Worst: O(N^2)** — on random or reverse-sorted input, roughly $N^2/2$ comparisons and up to $N^2/2$ swaps are needed (reverse-sorted is the true worst case: every single comparison triggers a swap).

Auxiliary space: O(1) — only the loop counters and the `swapped` flag; everything happens in-place via adjacent swaps.

EDGE CASES & PITFALLS

- **Omitting the early-exit flag entirely.** Without it, bubble sort always runs the full O(N^2) nested loop even on already-sorted input — a surprisingly common implementation gap that defeats the one property that makes bubble sort defensible to discuss at all.
- **Using `>=` instead of `>`.** Swapping on equal elements does unnecessary work and, more importantly, can break stability by needlessly reordering equal elements relative to each other.
- **Treating it as production-viable for anything beyond tiny N.** Bubble sort does *more* swaps than insertion sort for comparable work and has poor cache behavior relative to its O(N^2) peers — it's taught for pedagogical clarity (the simplest correct sort to reason about), essentially never chosen in production code where insertion sort would do strictly better.
- **Forgetting it's stable.** With strict `>`, equal elements are never swapped past each other, so bubble sort is stable — a fact worth stating explicitly, since the elementary sorts don't uniformly share this property (Selection Sort, Section 3, does not).

---

03 Selection Sort

The only sort in this guide with zero input-dependent variation in its comparison count — and a classic "is it stable?" trick question.

Overview & Working Principle

Selection sort repeatedly scans the **unsorted suffix** of the array to find its minimum element, then swaps that minimum into the front of the unsorted region. Unlike bubble or insertion sort, finding the minimum always requires scanning the *entire* remaining unsorted portion no matter what order the data is already in — there is no early-exit opportunity, which is exactly why its comparison count never varies with input order.

C++ Production Code

```cpp
// C++ program to implement Selection Sort
#include <bits/stdc++.h>
using namespace std;

void selectionSort(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {
        // Assume the current position holds
        // the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion
        // to find the actual minimum
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {
                // Update min_idx if a smaller
                // element is found
                min_idx = j;
            }
        }

        // Move minimum element to its
        // correct position
        swap(arr[i], arr[min_idx]);
    }
}

void printArray(vector<int> &arr) {
    for (int &val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};

    cout << "Original array: ";
    printArray(arr);

    selectionSort(arr);

    cout << "Sorted array: ";
    printArray(arr);

    return 0;
}
```

Source: GeeksforGeeks — Selection Sort Algorithm, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average / Worst: O(N^2), identically. The inner loop always scans the full remaining unsorted range to find the minimum — exactly `N(N-1)/2` comparisons every single time, regardless of whether the input starts sorted, reversed, or random. This is the only algorithm in this guide where best-case and worst-case are not just the same asymptotic class but the literal same comparison count.

Auxiliary space: O(1) — in-place, only a handful of index variables.

EDGE CASES & PITFALLS

- **Assuming it's stable — it isn't, in this standard form.** Swapping the found minimum directly into position `i` can jump it past an equal element that was already closer to the front, reversing their relative order. Example: `[3a, 3b, 1]` (subscripts marking otherwise-identical elements) becomes `[1, 3b, 3a]` after the first swap — `3a` and `3b` have swapped relative order. This is a genuinely common interview trick: many candidates assume all simple O(N^2) sorts are stable by default.
- **The one real advantage: minimal writes.** Selection sort performs at most N-1 swaps total — far fewer than bubble or insertion sort's potentially O(N^2) writes. This matters specifically when writes are expensive relative to comparisons (e.g. sorting data in flash memory with limited write endurance), one of the few scenarios where selection sort is a deliberate, defensible choice.
- **Using it believing it beats insertion sort.** It doesn't — insertion sort has the same O(N^2) worst case but a genuine O(N) best case on nearly-sorted data, which selection sort structurally cannot have.

---

04 Insertion Sort

The algorithm `std::sort` itself falls back to for small partitions — not a coincidence, given how it behaves on nearly-sorted data.

Overview & Working Principle

Insertion sort builds up a sorted region one element at a time: for each next element, it shifts already-sorted elements rightward to make room, inserting the new element into its correct

position within the sorted prefix — the same motion as sorting a hand of playing cards one card at a time. Because an already-sorted prefix needs *no* shifting for the next in-order element, insertion sort is naturally fast on data that's already close to sorted.

C++ Production Code

```cpp
// C++ program for implementation of Insertion Sort
#include <iostream>
using namespace std;

/* Function to sort array using insertion sort */
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

/* A utility function to print array of size n */
void printArray(int arr[], int n) {
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver method
int main() {
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);
    printArray(arr, n);

    return 0;
}
```

Source: GeeksforGeeks — Insertion Sort Algorithm, reproduced verbatim.

COMPLEXITY ANALYSIS

Best: O(N) — on already-sorted input, the `while` loop's condition `arr[j] > key` is false immediately every time, so each outer iteration does exactly one comparison and no shifting.

Average / Worst: O(N^2) — on reverse-sorted input specifically, every new element must shift all the way past every previously-sorted element, giving the full `N(N-1)/2` shifts.

Auxiliary space: O(1) — in-place, shifting within the same array.

EDGE CASES & PITFALLS

- **Why `std::sort` actually uses this (Section 12).** Introsort finishes small partitions (typically under ~16 elements) with insertion sort specifically because of its low constant-factor overhead at small sizes and its speed on the nearly-sorted data those small partitions often already are after quicksort's partitioning — this isn't a textbook footnote, it's load-bearing in real standard library implementations.
- **Using `>=` instead of `>` in the shift condition.** Same lesson as every other comparison-based sort in this guide — it still sorts correctly, but needlessly shifts past equal elements and can break stability.
- **Array-write shifting vs. linked-list insertion.** The array version above pays O(N) per insertion for shifting; on a doubly-linked list, the "shift" becomes an O(1) pointer splice once the insertion point is found — the search for that point is still O(N), but it's a meaningfully different cost profile worth knowing when asked to compare container choices.
- **Binary-search insertion point, linear shift.** A common "optimize this" follow-up: use binary search to find the insertion point in O(log N) instead of a linear scan — this reduces *comparisons* to O(N log N) total, but the shifting itself is still O(N) per insertion on a contiguous array, so the overall complexity remains O(N^2) in the worst case; the optimization only helps when comparisons are expensive relative to moves.

---

05 Shell Sort

Insertion sort generalized to move elements in big jumps first — the one algorithm in this guide whose complexity has no single textbook answer.

Overview & Working Principle

Shell sort runs insertion sort repeatedly over the array using a shrinking sequence of **gaps** instead of a gap of 1 from the start. With a large gap, far-apart elements get compared and swapped directly, moving badly-misplaced elements toward their final position in large jumps rather than one small shift at a time. As the gap shrinks — ending at exactly 1, which is plain insertion sort — the array is already much closer to sorted, so that final pass does far less work than a cold insertion sort would.

C++ Production Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

void shellSort(vector<int>& arr) {
    int n = arr.size();

    // Start with a large gap, then reduce it step by step
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Perform a "gapped" insertion sort for this gap size
        for (int i = gap; i < n; i++) {

            // Current element to be placed correctly
            int temp = arr[i];
            int j = i;

            // Shift elements that are greater than temp to make space
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            // Place temp in its correct location
            arr[j] = temp;
        }
    }
}

// Utility function to print the vector
void printArray(const vector<int>& arr) {
    for (int num : arr)
        cout << num << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {12, 34, 54, 2, 3};

    shellSort(arr);
    printArray(arr);

    return 0;
}
```

Source: GeeksforGeeks — ShellSort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best: O(N log N) with a well-chosen gap sequence on favorable input. **Average and worst case have no single closed-form answer** — they depend entirely on the gap sequence chosen, which is genuinely an open, empirically-studied area rather than a settled result. The simple "divide by 2" sequence shown above has a worst case of **$O(N^2)$**; the Hibbard and Sedgewick sequences improve the proven worst case to roughly **$O(N^{3/2})$**, and Knuth's sequence is commonly cited as giving an average case around **$O(N^{1.25})$** empirically.

Auxiliary space: O(1) — every pass is still an in-place insertion sort, just with a stride other than 1.

EDGE CASES & PITFALLS

- **The gap sequence is not a minor detail — it's the whole algorithm.** Unlike every other algorithm in this guide, there is no single "correct" Shell sort; the choice of gap sequence directly determines the proven complexity bounds, and this is a legitimate, still-studied research question rather than a solved problem with one textbook answer.
- **Forgetting the sequence must end at gap = 1.** If the shrinking sequence skips straight past 1 (an off-by-one in the loop condition or division), the array ends up only partially gap-sorted, not fully sorted — a subtle, easy-to-introduce correctness bug, not just a performance regression.
- **Not stable.** Comparisons and swaps happen between elements that are `gap` apart, which can and does reorder equal elements that started in different gap-groups relative to each other.
- **Reaching for it expecting Merge/Quick/Heap-level guarantees.** Shell sort is a genuine, practical improvement over plain insertion sort and is sometimes used in embedded/resource-constrained contexts for its O(1) space and simplicity, but it does not match the proven O(N log N) worst case of the comparison sorts in Sections 6–8 with simple gap sequences.

---

06 Merge Sort

The canonical divide-and-conquer sort: the one whose worst case you can prove on a whiteboard without hand-waving.

Overview & Working Principle

Merge sort recursively splits the array in half until each piece has one element (trivially sorted), then **merges** adjacent sorted pieces back together in linear time by repeatedly taking the smaller of the two pieces' front elements. Because the split is always exactly in half regardless of the data's actual order, the recursion depth is always exactly $\log_2 N$ — there is no input that produces a deeper or shallower call tree, which is precisely why merge sort has no "lucky" best case and no "unlucky" worst case the way quicksort does.

C++ Production Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(vector<int>& arr, int left,
                      int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp vectors
    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[],
    // if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```cpp
// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right){
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Driver code
int main(){
    vector<int> arr = {38, 27, 43, 10};
    int n = arr.size();

    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

Source: GeeksforGeeks — Merge Sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average / Worst: O(N log N), identically in all three cases. The recursion tree always has exactly $\log_2 N$ levels (the array is halved every time, unconditionally), and each level does O(N) total work across all its merge calls (every element is touched exactly once per level during merging) — $\log_2 N \times O(N) = O(N \log N)$, with no input-dependent variation at all.

Auxiliary space: O(N) — the merge step needs a temporary buffer to hold the merged result before copying it back, and while individual buffers are freed after each merge call returns, the standard analysis counts the largest single buffer alive at once, which is O(N) at the top-level merge.

EDGE CASES & PITFALLS

- **Integer overflow in the midpoint calculation.** Writing `mid = (left + right) / 2` can overflow `int` if `left` and `right` are both large (common with array indices near `INT_MAX` in competitive programming with huge arrays). Always write `mid = left + (right - left) / 2` instead — mathematically identical, never overflows.
- **Recursion stack overflow on very large inputs.** A naive recursive merge sort on tens of millions of elements can exhaust the call stack (depth O(log N) is small, but each frame's overhead adds up with large buffers); an iterative, bottom-up merge sort avoids recursion entirely for this reason in hardened production code.
- **Using `<` instead of `<=` in the merge comparison.** Using strict `<` when picking from the left vs. right half changes which equal element gets taken first and can silently break stability — the example above deliberately uses `<=` to prefer the left (earlier) element on ties.
- **Allocating a new buffer on every single merge call.** The code above does this for clarity, but it means O(log N) separate heap allocations total; production implementations typically allocate *one* N-sized buffer up front and reuse slices of it, avoiding repeated allocator overhead.

---

07 Quick Sort (Randomized)

The fastest sort in practice for most real data — and the one whose worst case you must be able to explain why randomization makes irrelevant.

Overview & Working Principle

Quick sort picks a **pivot** element, **partitions** the array so everything less than the pivot ends up to its left and everything greater ends up to its right (the pivot lands in its final sorted position in this single pass), then recursively sorts the two partitions. The **randomized** variant picks the pivot uniformly at random (swapping a random index to a fixed position, typically the last, before running a standard partition scheme) specifically to make the catastrophic $O(N^2)$ case — which for a *fixed* pivot rule (like "always pick the last element") is triggered by an already-sorted or reverse-sorted input — depend on the RNG instead of on any pattern an adversary (or an innocently-sorted input) could reliably construct.

C++ Production Code

```cpp
// C++ implementation QuickSort
// using Lomuto's partition Scheme.
#include <cstdlib>
#include <time.h>
#include <iostream>
using namespace std;

// This function takes last element
// as pivot, places
// the pivot element at its correct
// position in sorted array, and
// places all smaller (smaller than pivot)
// to left of pivot and all greater
// elements to right of pivot
int partition(int arr[], int low, int high)
{
    // pivot
    int pivot = arr[high];

    // Index of smaller element
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++)
    {
        // If current element is smaller
        // than or equal to pivot
        if (arr[j] <= pivot) {
             // increment index of
             // smaller element
            i++;

            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}

// Generates Random Pivot, swaps pivot with
// end element and calls the partition function
int partition_r(int arr[], int low, int high)
{
    // Generate a random number in between
    // low .. high
    srand(time(NULL));
    int random = low + rand() % (high - low);

    // Swap A[random] with A[high]
    swap(arr[random], arr[high]);

    return partition(arr, low, high);
}

/* The main function that implements QuickSort
arr[] --> Array to be sorted, low --> Starting index,
```

```
high --> Ending index */
void quickSort(int arr[], int low, int high)
{
    if (low < high) {
         /* pi is partitioning index,
        arr[p] is now
        at right place */
        int pi = partition_r(arr, low, high);

        // Separately sort elements before
        // partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

/* Function to print an array */
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        cout<<arr[i]<<" ";
}

// Driver Code
int main()
{
    int arr[] = { 10, 7, 8, 9, 1, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);

    quickSort(arr, 0, n - 1);
    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

Source: GeeksforGeeks — QuickSort using Random Pivoting, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average: O(N log N). When the pivot lands anywhere reasonably central (not necessarily exactly the median — even a consistent 25/75 split still gives O(N log N)), the recursion depth is O(log N) and each level does O(N) total partitioning work.

Worst: O(N^2). If the pivot is unluckily the minimum or maximum element on every single recursive call, one partition is always empty and the other has everything else, giving N levels of recursion each doing O(N) work. **Randomization doesn't eliminate this case — it makes the probability of hitting it on any given run astronomically small** (roughly `1/N!`-scale unlucky, versus a deterministic pivot rule where a specific, easy-to-construct input — e.g. already-sorted data with a "last element" pivot rule — triggers it *every single time*).

Auxiliary space: O(log N) expected (the recursion stack, when recursing on the smaller half and looping on the larger — a standard optimization) — but this bound assumes reasonably balanced partitions; the true worst case stack depth matches the O(N^2) time case at O(N).

EDGE CASES & PITFALLS

- **Deterministic pivot choice on sorted/reverse-sorted input.** This is the single most common way candidates accidentally reintroduce $O(N^2)$ behavior — "always pick the first/last element" degrades exactly on the inputs most likely to occur in practice (partially-sorted real-world data), which is the entire reason randomization (or median-of-three) exists.
- **Recursion stack overflow without the smaller-half optimization.** Naively recursing on *both* halves (rather than recursing on the smaller and looping/tail-recursing on the larger) turns the O(log N) expected stack depth into O(N) in the worst case — a real, production-observed stack overflow on large, adversarial, or simply unlucky inputs.
- **Not handling duplicate-heavy arrays.** The classic two-way Lomuto/Hoare partition above degrades toward $O(N^2)$ when most elements equal the pivot (every comparison goes the same way). A **three-way partition** (Dutch National Flag, grouping into less-than / equal-to / greater-than in one pass) fixes this and is the standard production answer for "many duplicate keys."
- **Not stable, and candidates often forget to say so.** Equal elements can and do get reordered during partitioning — if asked for a stable sort, quicksort is the wrong family entirely, not just an implementation detail to patch.
- `srand(time(NULL))` **called inside** `partition_r`**, on every recursive call.** Visible directly in the reference code above — reseeding the RNG from the system clock on every partition call is both wasteful (seeding has real cost) and statistically weak (multiple calls within the same clock second produce correlated, not independent, pivot choices). Production code seeds a generator *once* (e.g. a function-local `static std::mt19937`, seeded from `std::random_device`) and reuses it across the whole sort, rather than reseeding `rand()` from the wall clock on every partition.

---

08 Heap Sort

The only comparison sort here with a genuinely guaranteed O(N log N) worst case *and* O(1) auxiliary space — the trade-off is cache locality.

Overview & Working Principle

Heap sort first rearranges the array in-place into a **max-heap** (a complete binary tree, stored implicitly in the array, where every parent is ≥ its children) in O(N) time via a bottom-up **heapify**

pass. It then repeatedly swaps the root (always the current maximum) with the last unsorted element, shrinks the heap by one, and **sifts down** the new root to restore the heap property — N times total. Each swap places one more element in its final sorted position at the end of the array, so the array sorts itself in-place with no extra memory.

C++ Production Code

```cpp
#include <iostream>
#include<vector>
using namespace std;

// To heapify a subtree rooted with node i
void heapify(vector<int>& arr, int n, int i){
    // Initialize largest as root
    int largest = i;
    // left index = 2*i + 1
    int l = 2 * i + 1;
    // right index = 2*i + 2
    int r = 2 * i + 2;
    // If left child is larger than root
    if (l < n && arr[l] > arr[largest])
        largest = l;
    // If right child is larger than largest so far
    if (r < n && arr[r] > arr[largest])
        largest = r;
    // If largest is not root
    if (largest != i) {
        swap(arr[i], arr[largest]);
        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}

// Main function to do heap sort
void heapSort(vector<int>& arr){
    int n = arr.size();
    // Build heap (rearrange vector)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // One by one extract an element from heap
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        swap(arr[0], arr[i]);
        // Call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

int main(){
    vector<int> arr = { 9, 4, 3, 8, 10, 2, 5 };
    heapSort(arr);
    for (int i = 0; i < arr.size(); ++i)
        cout << arr[i] << " ";
}
```

Source: GeeksforGeeks — Heap Sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Build phase: O(N) — this is the famous non-obvious result: although there are N sift-down calls, most nodes are near the bottom of the tree and sift down only a short distance, so the sum telescopes to O(N), not the naive O(N log N) upper bound you'd get from assuming every node sifts the full tree height.

Extraction phase: O(N log N) — N extractions, each requiring one O(log N) sift-down to restore the heap property, with no input-dependent variation (the tree shape only depends on N, never on the data's order), which is exactly why **best, average, and worst case are all identically O(N log N)**.

Auxiliary space: O(1) — every operation (swap, sift-down) works in-place on the original array; no recursion-proportional stack growth either, since sift-down's recursion depth is bounded by the heap's height, O(log N), and that's conventionally treated as part of the O(1) in-place classification for this algorithm (some stricter analyses call it O(log N) for the implicit recursion stack — know both framings).

EDGE CASES & PITFALLS

- **Starting the build phase from the wrong index.** The build loop must start at `n/2 - 1` (the last node with at least one child), *not* at `n-1` (the last leaf) — leaves have no children to sift below them, so starting there wastes N/2 no-op calls and, if done wrong, can leave the heap property unestablished for internal nodes.
- **Poor cache locality in practice.** Despite matching quicksort's average-case time complexity on paper, heap sort is almost always slower in real benchmarks — sibling/parent-child array indices (`2i+1` , `2i+2`) jump around memory non-sequentially, unlike quicksort's largely sequential partition scans, which is why heap sort is chosen for its *guarantees* (worst case, space), not raw speed.
- **Not stable.** The swap-heavy extraction process routinely reorders equal elements relative to each other.
- **Off-by-one in the child index formulas.** Using 1-indexed child formulas (`2i` , `2i+1`) on a 0-indexed array (which needs `2i+1` , `2i+2`) is a frequent, silent correctness bug — it compiles and often "mostly" works on small test cases while subtly corrupting the heap property on others.

09 Counting Sort

Not a comparison sort at all — and the first of three algorithms in this guide that legitimately beat the O(N log N) comparison-sort lower bound.

Overview & Working Principle

Counting sort exploits a constraint comparison sorts don't assume: that keys are **integers within a known, bounded range** `[0, K]`. It counts how many times each distinct key value occurs, converts those counts into prefix sums (so each count becomes "how many elements belong at or before this key"), then places every element directly into its final position in one pass using those prefix sums — no pairwise comparisons between elements ever happen, which is exactly how it sidesteps the `Ω(N log N)` lower bound that applies to comparison-based sorting specifically.

C++ Production Code

```cpp
#include <iostream>
#include <vector>
using namespace std;

vector<int> countsort(vector<int>& arr) {
    int n = arr.size();
    // find the maximum element
    int maxval = 0;
    for (int i = 0; i < n; i++)
        maxval = max(maxval, arr[i]);
    // create and initialize cntArr array
    vector<int> cntArr(maxval + 1, 0);
    // count frequency of each element
    for (int i = 0; i < n; i++)
        cntArr[arr[i]]++;
    // compute prefix sum
    for (int i = 1; i <= maxval; i++)
        cntArr[i] += cntArr[i - 1];
    // build output array
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--) {
        ans[cntArr[arr[i]] - 1] = arr[i];
        cntArr[arr[i]]--;
    }
    return ans;
}

int main() {
    vector<int> arr = {2,5,3,0,2,3,0,3};
    vector<int> ans = countsort(arr);
    for (int x : ans)
        cout << x << " ";
    return 0;
}
```

Source: GeeksforGeeks — Counting Sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average / Worst: O(N + K), identically — the algorithm does exactly one pass over the N input elements to count, one pass over the K possible key values to build prefix sums, and one more pass over the N elements to place them, with zero input-order dependence anywhere in that process.

Auxiliary space: O(N + K) — the `count` array costs O(K), and the `output` array costs O(N); both are mandatory, not implementation artifacts to optimize away.

Why this isn't a contradiction of the O(N log N) lower bound: that bound is specifically proven for algorithms that can only learn information about the input by *comparing pairs of elements*. Counting sort never compares two elements to each other at all — it uses each element's value directly as an array index, which is extra information a pure comparison model doesn't have access to.

EDGE CASES & PITFALLS

- **K much larger than N.** If keys range over, say, `[0, 10^9]` but there are only 100 elements, the O(N + K) bound becomes O(K)-dominated and catastrophically worse than O(N log N) — counting sort is only a good choice when K is known to be reasonably close to O(N).
- **Negative keys.** The algorithm as written assumes non-negative integer keys usable directly as array indices; negative keys need an offset shift (subtract the minimum value first) before counting, and forgetting this is a common out-of-bounds-access bug.
- **Forgetting the stability-preserving reverse iteration.** Placing elements by iterating the input *forward* instead of backward during the final placement pass still produces a correctly *sorted* array, but silently breaks stability (equal keys end up reversed relative to their input order) — a frequent, easy-to-miss detail when candidates reconstruct this from memory.
- **Using it on floating-point or non-integer keys.** Counting sort fundamentally requires discrete, indexable keys — it cannot be applied to `double` values without first bucketing/discretizing them, which is a different algorithm (Bucket Sort, Section 11) with its own trade-offs. It's also the exact building block Radix Sort (Section 10) uses, one digit at a time.

10 Radix Sort

Counting sort applied repeatedly, one digit at a time — correctness here hinges entirely on stability, not just speed.

Overview & Working Principle

Radix sort sorts integers (or fixed-width keys generally) digit by digit, starting from the **least significant digit** and working up to the most significant, using a stable sort — almost always Counting Sort (Section 9) — as the subroutine for each digit position. The correctness argument is subtle and important: after sorting by the least-significant digit, then stably sorting that result by the next digit up, elements that tie on the current digit retain their relative order from the previous (less significant) pass — so by the time the most significant digit's pass finishes, the array is fully sorted, with every digit position having contributed correctly to the final order.

C++ Production Code

```cpp
#include <iostream>
using namespace std;

// A utility function to get maximum value in arr[]
int getMax(int arr[], int n) {
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

// A function to do counting sort of arr[] according to
// the digit represented by exp.
void countSort(int arr[], int n, int exp) {
    int output[n]; // output array
    int i, count[10] = {0};

    // Store count of occurrences in count[]
    for (i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Change count[i] so that count[i] now contains actual
    // position of this digit in output[]
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array
    for (i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the output array to arr[], so that arr[] now
    // contains sorted numbers according to current digit
    for (i = 0; i < n; i++)
        arr[i] = output[i];
}

// The main function to that sorts arr[] of size n using
// Radix Sort
void radixsort(int arr[], int n) {
    // Find the maximum number to know number of digits
    int m = getMax(arr, n);

    // Do counting sort for every digit. Note that instead
    // of passing digit number, exp is passed. exp is 10^i
    // where i is current digit number
    for (int exp = 1; m / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

// A utility function to print an array
void print(int arr[], int n) {
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
    cout << endl;
}

// Driver Code
int main() {
    int arr[] = { 170, 45, 75, 90, 802, 24, 2, 66 };
    int n = sizeof(arr) / sizeof(arr[0]);
    radixsort(arr, n);
    print(arr, n);
    return 0;
}
```

Source: GeeksforGeeks — Radix Sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average / Worst: O(d × (N + b)), identically — where `d` is the number of digits in the largest key and `b` is the base used (10 above; often 256 for byte-wise radix sort operating directly on a key's binary representation). For fixed-width keys — a 32-bit integer has at most 10 decimal digits, or exactly 4 passes at byte granularity — `d` is a small constant, making this effectively **O(N)** in practice. Like Counting Sort, this is not a comparison sort, so it is not bound by the O(N log N) comparison-sort lower bound.

Auxiliary space: O(N + b) per digit pass — the same count/output buffers Counting Sort needs, reused across passes.

EDGE CASES & PITFALLS

- **Using an unstable sort as the per-digit subroutine.** This is the single most important correctness requirement of radix sort, not a performance nicety — if the per-digit sort isn't stable, elements that tied on a less-significant digit can end up in the wrong relative order after a later, more-significant-digit pass, producing a genuinely incorrectly-sorted final array, not just a slow one.
- **Negative numbers.** Direct digit extraction via `(x / exp) % 10` assumes non-negative values; negative keys need explicit handling — typically sorting the magnitudes and handling sign separately, or using a biased/offset representation.
- **One large outlier inflating every pass.** The number of passes `d` is determined by the *largest* element's digit count — a single unusually large value (e.g. one 9-digit number among a million small 2-digit numbers) forces every element through several otherwise-unnecessary extra passes.
- **Doesn't generalize the way comparison sorts do.** Radix sort works cleanly for fixed-width integers and strings (sorted character position by character position, from the right for fixed-length strings), but adapting it to arbitrary comparable types, floating-point numbers, or variable-length keys requires real, non-obvious engineering — unlike a comparison sort, which only ever needs a working `<`.

---

11 Bucket Sort

The one algorithm in this guide whose average-case bound depends on an assumption about your data, not just its size.

Overview & Working Principle

Bucket sort distributes elements into a fixed number of **buckets** based on their value, sorts each bucket individually (commonly with insertion sort, since well-distributed buckets are expected to hold only a handful of elements each), then concatenates the buckets back together in order. Its average-case linear-time analysis specifically assumes the input is **roughly uniformly distributed** across its range — unlike Counting Sort or Radix Sort, which make no assumption about the data's distribution at all, only about its structure (bounded integer keys).

C++ Production Code

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to sort a bucket using insertion sort
void insertionSort(vector<float>& bucket) {
    for (int i = 1; i < bucket.size(); i++) {
        float key = bucket[i];
        int j = i - 1;
        while (j >= 0 && bucket[j] > key) {
            bucket[j + 1] = bucket[j];
            j--;
        }
        bucket[j + 1] = key;
    }
}

// Function to sort arr[] of size n using bucket sort
void bucketSort(float arr[], int n) {

    // Create n empty buckets
    vector<float> b[n];

    // Put array elements into different buckets
    for (int i = 0; i < n; i++) {
        int bi = n * arr[i]; // Index in the bucket
        b[bi].push_back(arr[i]);
    }

    // Sort individual buckets using insertion sort
    for (int i = 0; i < n; i++) {
        insertionSort(b[i]);
    }

    // Concatenate all buckets into arr[]
    int index = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < b[i].size(); j++) {
            arr[index++] = b[i][j];
        }
    }
}

// Driver code
int main() {
    float arr[] = {0.897, 0.565, 0.656, 0.1234, 0.665, 0.3434};
    int n = sizeof(arr) / sizeof(arr[0]);
    bucketSort(arr, n);

    cout << "Sorted array is \n";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Source: GeeksforGeeks — Bucket Sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average: O(N + K) where `K` is the number of buckets — *assuming* a roughly uniform input distribution, each bucket receives O(1) elements on average, so sorting each bucket costs O(1) and the N total elements across K buckets sum to O(N + K) overall.

Worst: O(N^2) (or O(N log N), depending on the per-bucket sort used) — if the distribution is adversarial and every element lands in the same bucket, bucket sort degenerates into running whatever the per-bucket sort is on the entire array at once, with none of the distribution benefit.

Auxiliary space: O(N + K) — the bucket structures themselves, holding all N elements across K buckets.

EDGE CASES & PITFALLS

- **The average-case bound is conditional, and candidates often state it unconditionally.** A correct answer explicitly names the uniform-distribution assumption — unlike Counting Sort or Radix Sort, bucket sort is not reliably linear for arbitrary data, only for data that genuinely resembles the distribution the bucket boundaries were chosen for.
- **Silent performance degradation from bad bucket boundaries.** Choosing too few buckets, or boundaries that don't match the real data's distribution (e.g. assuming uniform [0,1) data but actually receiving heavily-skewed real-world measurements), degrades performance toward the worst case with no error or warning — worth explicitly checking input characteristics, or using data-dependent bucket boundaries, before relying on this in production.
- **Requires a known value range.** Mapping a value to a bucket index needs the value's range up front (here, `[0, 1)` floats) — unlike comparison sorts, which need nothing beyond a working `<`.
- **Stability depends on the per-bucket sort.** Bucket sort is stable overall *only if* both the bucket-assignment step preserves input order within each bucket (trivially true with `push_back`, as above) *and* the per-bucket sort itself is stable — swapping in an unstable per-bucket sort silently removes the overall stability guarantee.

12 `std::sort`

The default, almost-always-correct choice for sorting a random-access range in C++.

Overview & Working Principle

`std::sort` is not one algorithm — it's a hybrid called **introsort** (introspective sort). It starts as quicksort for speed, but tracks its own recursion depth; if that depth exceeds roughly `2 × log₂(N)` (a sign the pivot choices are degenerating toward $O(N^2)$), it switches that partition to **heapsort** (Section 8) to guarantee the O(N log N) worst case. Small partitions (typically under 16 elements) are finished off with **insertion sort** (Section 4), which has lower constant-factor overhead than recursive partitioning at that size. This is why the Standard mandates O(N log N) worst-case as of C++11 — the heapsort fallback is what makes that guarantee possible.

C++ Production Code

```cpp
#include <algorithm>
#include <array>
#include <functional>
#include <iostream>
#include <string_view>

int main()
{
    std::array<int, 10> s{5, 7, 4, 2, 8, 6, 1, 9, 0, 3};

    auto print = [&s](std::string_view const rem)
    {
        for (auto a : s)
            std::cout << a << ' ';
        std::cout << ": " << rem << '\n';
    };

    std::sort(s.begin(), s.end());
    print("sorted with the default operator<");

    std::sort(s.begin(), s.end(), std::greater<int>());
    print("sorted with the standard library compare function object");

    struct
    {
        bool operator()(int a, int b) const { return a < b; }
    }
    customLess;

    std::sort(s.begin(), s.end(), customLess);
    print("sorted with a custom function object");

    std::sort(s.begin(), s.end(), [](int a, int b)
    {
        return a > b;
    });
    print("sorted with a lambda expression");
}
```

Source: cppreference.com — std::sort, reproduced verbatim.

COMPLEXITY ANALYSIS

Best / Average / Worst: O(N log N) in every case, guaranteed since C++11. The quicksort phase gives the excellent average-case constant factor (good cache locality, few comparisons); the depth-limited heapsort fallback is what converts the theoretical quicksort worst case of $O(N^2)$ into a hard O(N log N) ceiling — it only ever triggers on the specific partitions that are degenerating, not the whole array.

Auxiliary space: O(log N) — this is purely the recursion stack depth for the quicksort phase (implementations typically recurse on the smaller partition and loop on the larger one specifically to bound this to O(log N), not O(N)). Heapsort and insertion sort both run in-place with no extra allocation.

EDGE CASES & PITFALLS

- **Not stable.** Equal elements can be reordered relative to each other. If you need to preserve original relative order among equal keys (e.g. sorting log entries by severity, then wanting same-severity entries to stay in timestamp order), `std::sort` will silently violate that — use `std::stable_sort` instead.
- **Comparator must satisfy Strict Weak Ordering.** Using `<=` instead of `<` in a custom comparator is undefined behavior — it can segfault or infinite-loop on libstdc++/MSVC's introsort implementation. Covered in full in Section 15.
- **Iterator invalidation if the comparator mutates the container.** A comparator that modifies elements of the same range it's sorting (rather than just comparing them) is undefined behavior — comparators must be pure, read-only predicates.
- **Requires random-access iterators.** You cannot `std::sort` a `std::list` directly (it only provides bidirectional iterators) — use `list::sort()`, its own member function, instead.

---

13 `std::stable_sort`

Same interface as `std::sort`, with one guarantee added: equal elements never change relative order.

Overview & Working Principle

`std::stable_sort` is typically implemented as a **bottom-up merge sort**. Most implementations (libstdc++, libc++) attempt to allocate a temporary buffer of size N up front; if that succeeds, the algorithm proceeds as a classic merge sort (Section 6) using that buffer and achieves O(N log N) worst case. If the allocation fails (extremely rare in practice, but the Standard must account for it), the implementation falls back to an adaptive, in-place merge algorithm that trades time for the missing space, degrading to $O(N \log^2 N)$.

C++ Production Code

```cpp
#include <algorithm>
#include <array>
#include <iostream>
#include <string>
#include <vector>

struct Employee
{
    int age;
    std::string name; // Does not participate in comparisons
};

bool operator<(const Employee& lhs, const Employee& rhs)
{
    return lhs.age < rhs.age;
}

#if __cpp_lib_constexpr_algorithms >= 202306L
consteval auto get_sorted()
{
    auto v = std::array{3, 1, 4, 1, 5, 9};
    std::stable_sort(v.begin(), v.end());
    return v;
}
static_assert(std::ranges::is_sorted(get_sorted()));
#endif

int main()
{
    std::vector<Employee> v{{108, "Zaphod"}, {32, "Arthur"}, {108, "Ford"}};

    std::stable_sort(v.begin(), v.end());

    for (const Employee& e : v)
        std::cout << e.age << ", " << e.name << '\n';
}
```

Source: cppreference.com — std::stable_sort, reproduced verbatim.

Output — note Zaphod (original index 0) prints before Ford (original index 2), even though both have `age == 108`; their relative order from the input is preserved:

```
32, Arthur
108, Zaphod
108, Ford
```

COMPLEXITY ANALYSIS

Best / Average: O(N log N), same reasoning as merge sort (Section 6) — the array is always recursively halved regardless of input order, so there's no faster "already sorted" best case the way insertion-sort-based algorithms have.

Worst: O(N log N) with sufficient memory, **O(N $\log^2$ N)** if the allocation fails and it falls back to in-place merging — the extra log N factor comes from the in-place merge needing an additional rotation step per merge level to make room without a buffer.

Auxiliary space: O(N) — unlike `std::sort`, this is the one guarantee you pay for directly: a full temporary buffer the size of the input.

EDGE CASES & PITFALLS

- **Memory pressure on large datasets.** The O(N) buffer is a real, often-overlooked cost — sorting a multi-gigabyte dataset with `stable_sort` can momentarily double peak memory usage; this is frequently the actual reason a production job OOMs, not the algorithm's time complexity.
- **Assuming it's always O(N log N).** The degraded O(N $\log^2$ N) fallback path is rarely hit, but a correct answer should mention it exists — interviewers specifically probe this to separate candidates who memorized a table from candidates who understand why the row has two different worst-case numbers depending on context.
- **Reaching for it when stability doesn't actually matter.** If nothing in your data ever compares equal (e.g. sorting by a unique ID), stability is a no-op guarantee you're paying O(N) space for nothing — use `std::sort`.

---

14 Timsort (Conceptual Overview)

Not part of the C++ Standard Library — but the single most practically important sorting algorithm in software today, and a frequent comparison point against `std::stable_sort`.

NOT A C++ STANDARD ALGORITHM

Timsort powers Python's `sorted()` / `list.sort()` and Java's `Arrays.sort()` for object arrays (primitive arrays in Java still use a dual-pivot quicksort variant). It is **not** part of the C++ Standard Library — `std::stable_sort` (Section 13) uses a straightforward merge sort, not Timsort. It's included here because "compare Timsort to `std::stable_sort`" and "name a real-world hybrid sorting algorithm" are both genuinely common interview questions.

Overview & Working Principle

Timsort is a hybrid of merge sort and insertion sort, designed by Tim Peters specifically to exploit **existing order** in real-world data, which a textbook merge sort's rigid always-halve-in-the-middle structure completely ignores. It scans the input for naturally-occurring ascending or descending **runs** (already-sorted subsequences), extends any run shorter than a minimum threshold up to that threshold using insertion sort (cheap, since the run is already nearly sorted), then merges the resulting runs together using a carefully tuned merge strategy that preserves stability and keeps the merge step's overhead low.

```
// CONCEPTUAL pseudocode -- not compilable C++, no C++ standard implementation exists
function timsort(arr):
    runs = findNaturalRuns(arr)             // scan for already-ascending/descending stretches
    for run in runs:
        if run.length < MIN_RUN:
            insertionSortExtend(run, MIN_RUN) // pad short runs up to MIN_RUN via insertion sort
    while len(runs) > 1:
        mergeRuns(runs)                       // merge adjacent runs, stably, using a tuned merge order
    return runs[0]
```

COMPLEXITY ANALYSIS

Best: O(N) — if the entire input is already a single ascending (or descending) run, Timsort detects this directly and does essentially no further work, a best case classic merge sort structurally cannot have.

Average / Worst: O(N log N) — the same family bound as merge sort, since the underlying merge step is still a standard stable merge.

Auxiliary space: O(N) — merging, as always, needs a buffer proportional to the data being merged.

WHY THIS IS WORTH KNOWING EVEN THOUGH IT'S NOT IN C++

- **It's the canonical answer to "name a real-world production hybrid sort."** Unlike the textbook algorithms above, Timsort was built by directly analyzing real-world data (which is very often partially sorted — nearly-sorted logs, a mostly-sorted list after one new insertion) rather than optimizing purely for worst-case asymptotic behavior.
- **It's the sharpest contrast with `std::stable_sort`.** Both guarantee O(N log N) worst case, O(N) space, and stability — but classic merge sort (what `std::stable_sort` uses) is deliberately *oblivious* to existing order for the cleanliness of its recursive proof, while Timsort is explicitly **adaptive**, exploiting existing order as its primary real-world performance win.
- **It illustrates that "state the complexity" isn't the end of a real answer.** Two algorithms can share an identical Big-O row in the cheat sheet (Section 1) and still differ enormously in real, measured performance on real data — the constant factors and adaptivity the asymptotic notation hides are often the actual point of a senior-level follow-up question.

---

15 Custom Comparators & Strict Weak Ordering

The single most common source of real segfaults and undefined behavior in otherwise-correct-looking C++ sorting code.

Three ways to write a comparator

```cpp
// 1. FUNCTION POINTER -- a plain free function, passed by its address
bool byDescending(int a, int b) { return a > b; }
std::sort(v.begin(), v.end(), byDescending);

// 2. FUNCTOR -- a struct/class with operator() -- can carry STATE, and is
// more likely to be inlined by the compiler than a function pointer
struct ByDistanceFrom {
    int origin;
    bool operator()(int a, int b) const {
        return std::abs(a - origin) < std::abs(b - origin);
    }
};
std::sort(v.begin(), v.end(), ByDistanceFrom{50});

// 3. LAMBDA -- the modern default; captures context inline, no named type needed
int origin = 50;
std::sort(v.begin(), v.end(),
    [origin](int a, int b) { return std::abs(a - origin) < std::abs(b - origin); });
```

All three are functionally equivalent to `std::sort` — it's templated on the comparator type and calls it the same way regardless of which form you pass. The practical differences are about **state** (a plain function pointer can't carry data without a global; a functor or lambda can) and **optimization** (functors and lambdas have a unique type per instantiation, which the compiler can far more reliably inline than an indirect call through a function pointer — in hot sorting code this is a measurable, real performance difference, not just style).

Strict Weak Ordering: the rule every comparator must obey

Every comparator passed to a C++ sorting or ordering algorithm must implement a **Strict Weak Ordering** relation `comp(a, b)` meaning "`a` is strictly less than `b`." Concretely, it must satisfy:

- **Irreflexivity:** `comp(a, a)` must always be `false` — an element is never less than itself.
- **Asymmetry:** if `comp(a, b)` is `true`, then `comp(b, a)` must be `false`.
- **Transitivity:** if `comp(a, b)` and `comp(b, c)` are both `true`, then `comp(a, c)` must be `true` too.
- **Transitivity of equivalence:** if `a` and `b` are "equivalent" (neither is less than the other) and `b` and `c` are equivalent, then `a` and `c` must be equivalent too.

THE `<=` BUG: CODE THAT COMPILES, PASSES SMALL TESTS, AND SEGFAULTS IN PRODUCTION

```
// WRONG -- violates irreflexivity: comp(5, 5) returns true, which is a lie
std::sort(v.begin(), v.end(),
    [](int a, int b) { return a <= b; });    // << the bug is exactly here
```

This single-character mistake (`<=` instead of `<`) is the most common Strict Weak Ordering violation in real codebases. Internally, introsort's partitioning and heapsort's sift-down both rely on the mathematical guarantees above to reason about element positions without re-checking them — when `comp(x, x)` can return `true` , those internal invariants break, and depending on the standard library implementation, input size, and exact data, the observed failure ranges from a silently wrong (but not crashing) sort order, to an out-of-bounds array access and a genuine **segmentation fault** — notoriously, often only on *larger* inputs that happen to exercise the code path where the corrupted invariant actually gets dereferenced out of bounds, which is exactly why this bug so often passes small unit tests and then crashes in production.

THE CORRECT VERSION

```
// RIGHT -- strict less-than; comp(5, 5) correctly returns false
std::sort(v.begin(), v.end(),
    [](int a, int b) { return a < b; });
```

A SUBTLER SWO VIOLATION: INCONSISTENT MULTI-KEY COMPARATORS

```
// WRONG -- not transitive: compares by `age`, but falls back to comparing
// `name` only SOMETIMES, based on an unrelated, inconsistent condition
bool buggyCompare(const Person& a, const Person& b) {
    if (a.age != b.age) return a.age < b.age;
    if (a.active)       return a.name < b.name;  // only tie-breaks SOMETIMES
    return false;
}
```

This compiles fine and often "looks" sorted on casual inspection, but breaks transitivity for inactive elements with equal ages — the algorithm's internal assumptions about ordering silently desynchronize from reality, which can manifest as elements that are "almost" sorted, or, again, an out-of-bounds access deep inside the library's partitioning logic depending on implementation and input.

HOW TO DEFEND THIS IN AN INTERVIEW

State the rule by name (Strict Weak Ordering), name the specific property a buggy comparator violates (irreflexivity, transitivity, etc. — don't just say "it's wrong"), and explain the *mechanism* of the failure: the sorting algorithm's internal logic (partition boundaries, heap invariants, merge assumptions) relies on these mathematical properties holding unconditionally, so violating them corrupts the algorithm's own bookkeeping, not just the final output order.

---

16 Sorting Custom Data Structures

Where the theory above becomes the code you actually write day to day.

`std::vector<std::pair<int, int>>`

`std::pair` already has a built-in `operator<` that compares lexicographically — first by `.first`, and only by `.second` to break ties — so `std::sort` with no comparator at all already does "sort by first element, then second" for free.

```cpp
#include <vector>
#include <algorithm>

std::vector<std::pair<int, int>> intervals = {{3, 1}, {1, 5}, {1, 2}, {2, 4}};

// default operator< : sorts by .first ascending, ties broken by .second ascending
std::sort(intervals.begin(), intervals.end());
// result: {1,2}, {1,5}, {2,4}, {3,1}

// sort by SECOND element instead -- needs an explicit comparator
std::sort(intervals.begin(), intervals.end(),
    [](const auto& a, const auto& b) { return a.second < b.second; });
```

Custom `struct Student`, multiple criteria

Real-world sorting almost always means "sort by field A, and for ties, by field B" — the standard pattern is a chain of `if` statements that only ever falls through to the next field when the current one is exactly equal.

```cpp
#include <vector>
#include <algorithm>
#include <string>

struct Student {
    std::string name;
    int gradeLevel;
    double gpa;
};

std::vector<Student> students = { /* ... */ };

// primary: gradeLevel ascending. tie-break: gpa descending. final tie-break: name ascending
std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        if (a.gradeLevel != b.gradeLevel) return a.gradeLevel < b.gradeLevel;
        if (a.gpa != b.gpa)               return a.gpa > b.gpa;        // descending
        return a.name < b.name;                                        // final, deterministic tie-break
    });

// the same logic, expressed via std::tie -- concise, and easy to verify SWO-correct
// because std::tuple/std::tie's operator< already does exactly this lexicographic chain
#include <tuple>
std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        return std::tie(a.gradeLevel, b.gpa, a.name)
             < std::tie(b.gradeLevel, a.gpa, b.name);   // note the swapped gpa order for DESC
    });
```

EDGE CASES & PITFALLS

- **Comparing floating-point fields with `==` for the "is this a tie" check.** `a.gpa != b.gpa` as a tie-break guard is exact-equality floating-point comparison — two GPAs that are conceptually "the same" but differ in the last bit of precision (e.g. one computed as `3.7` and another as a running average that lands on `3.6999999999999997`) will be treated as a real difference, giving a technically-valid but possibly unintended ordering. For derived/computed floating-point fields specifically, consider a small epsilon-aware comparison instead of exact `!=`.
- **No final, deterministic tie-break field.** If two `Student`s are identical on every field the comparator checks, `std::sort` (unstable) may place them in either order on different runs/platforms/library versions — always end a multi-criteria comparator with a field guaranteed to be unique (an ID, or as a last resort, `std::tie` over every field) if order-reproducibility matters for your use case.
- **Forgetting `const&` parameters.** Taking `Student` by value in the comparator means every single comparison during the sort copies the whole struct — for anything beyond a few trivial fields, this is a real, easily-overlooked performance cost multiplied by O(N log N) comparisons.
- **The `std::tie` version's descending field is easy to get backwards.** To sort a field descending while using `std::tie`'s lexicographic `operator<`, you must swap that one field's operands between the two `tie()` calls (as shown above) — forgetting this silently sorts every field ascending instead.

---

17 Universal Edge Cases Checklist

Run every sorting implementation — yours or a library's you're evaluating — against this list before calling it done. These are the inputs interviewers and competitive-programming judges specifically construct to break naive implementations.

- ☐ **Empty array** — does the function handle `size() == 0` without reading out of bounds?
- ☐ **Single element** — the smallest non-trivial base case; recursive algorithms especially must terminate correctly here.
- ☐ **Already sorted** — the classic quicksort worst-case trigger for naive (non-randomized, non-median-of-three) pivot selection; also the case every elementary sort's "best case" claim depends on.
- ☐ **Reverse sorted** — often an even worse trigger than "already sorted" for a fixed first/last-element pivot rule, and the true worst case for insertion sort.
- ☐ **All duplicate elements** — stresses Strict Weak Ordering correctness directly, and triggers quicksort's duplicate-heavy $O(N^2)$ degradation without a three-way partition.
- ☐ **Negative numbers** — breaks counting sort and radix sort's direct array-indexing unless an offset shift is applied first.
- ☐ `INT_MAX` / `INT_MIN` **present** — the classic integer-overflow trigger for midpoint calculations and any arithmetic performed directly on key values (e.g. a difference-based comparator); also inflates Radix Sort's digit-pass count.
- ☐ **Very large N** — surfaces recursion-stack-depth issues (naive merge/quick sort) and O(N) auxiliary-space costs (`stable_sort` , merge sort, Timsort) that are invisible on small test inputs.
- ☐ **Highly skewed / non-uniform distribution** — specifically breaks Bucket Sort's average-case assumption; worth testing even when the "happy path" distribution looks fine.
- ☐ **Keys with a huge range relative to N** — breaks Counting Sort and Radix Sort's linear-time assumption (O(K) or O(d) dominates when the key range or digit count is large relative to N).

How to Export This Guide to PDF

This page is already print-formatted — code blocks wrap instead of clipping, and tables resize to the page width.

1. Click the **Save as PDF** button in the top bar (uses your browser's native print engine), or press `Ctrl/Cmd+P`.
2. Set **Destination** to *Save as PDF*, and enable **Background graphics** so the comparison-table cells and callout boxes keep their color.
3. Default margins work well with this layout.

C++ Sorting Algorithms & Custom Sorting Guide · targets C++17/C++20 · 13 algorithms: Bubble, Selection, Insertion, Shell, Merge, Quick (Randomized), Heap, Counting, Radix, Bucket, std::sort, std::stable_sort, Timsort (conceptual)